Welcome to the ADI Interview Prep Series: Week 3

 Fall 2025

slidesmania.com

Overview

Week 1: Arrays + Hashing

Week 2: Two Pointers + Sliding Window

Week 3: Binary Search + Trees

Week 4: Stack + Linked List

Week 5: Heap + Priority Queue

Week 6: Graphs

Week 7: Dynamic Programming + Greedy

slidesmania.com

TODAY'S PLAN:

01

Review of Binary Search

02

Problem 1: Sqrt(x)

[15 minutes]

03

Problem 2: Lowest Common Ancestor of Binary Search Tree

[20 minutes]

04

Problem 3: Capacity to Ship Packages Within D Days

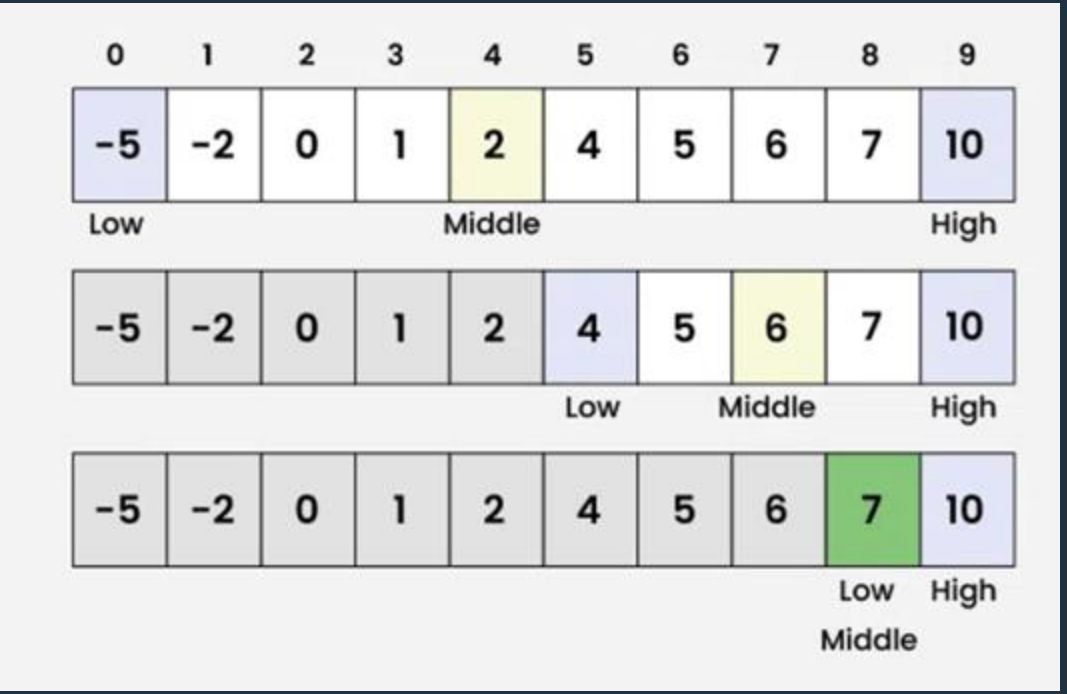
[30 minutes]

05

06

slidesmania.com

Binary Search



slidesmania.com

Binary Search

```
class BinarySearch {

    // Returns index of x if it is present in arr[].
    int binarySearch(int arr[], int x)
    {
        int low = 0, high = arr.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;

            // Check if x is present at mid
            if (arr[mid] == x)
                return mid;

            // If x greater, ignore left half
            if (arr[mid] < x)
                low = mid + 1;

            // If x is smaller, ignore right half
            else
                high = mid - 1;
        }

        // If we reach here, then element was
        // not present
        return -1;
    }
```

slidesmania.com

PROBLEM ONE: Sqrt(X)

You are given a non-negative integer x, return the square root of x rounded down to the nearest integer. The returned integer should be non-negative as well.

You must not use any built-in exponent function or operator.

For example, do not use pow(x, 0.5) in c++ or x ** 0.5 in python.

Example 1:

Input: x = 9

Output: 3

Example 2:

Input: x = 13

Output: 3

slidesmania.com

PROBLEM ONE: Sqrt(X)

You are given a non-negative integer x, return the square root of x rounded down to the nearest integer. The returned integer should be non-negative as well.

You must not use any built-in exponent function or operator.

For example, do not use pow(x, 0.5) in c++ or x ** 0.5 in python.

The brute force approach would be iterating through all values from 1 to the integer x, stopping whenever we reach the square root. Is there perhaps a faster way to accomplish this?

slidesmania.com

Python | **Java** | C++ | JavaScript | C#

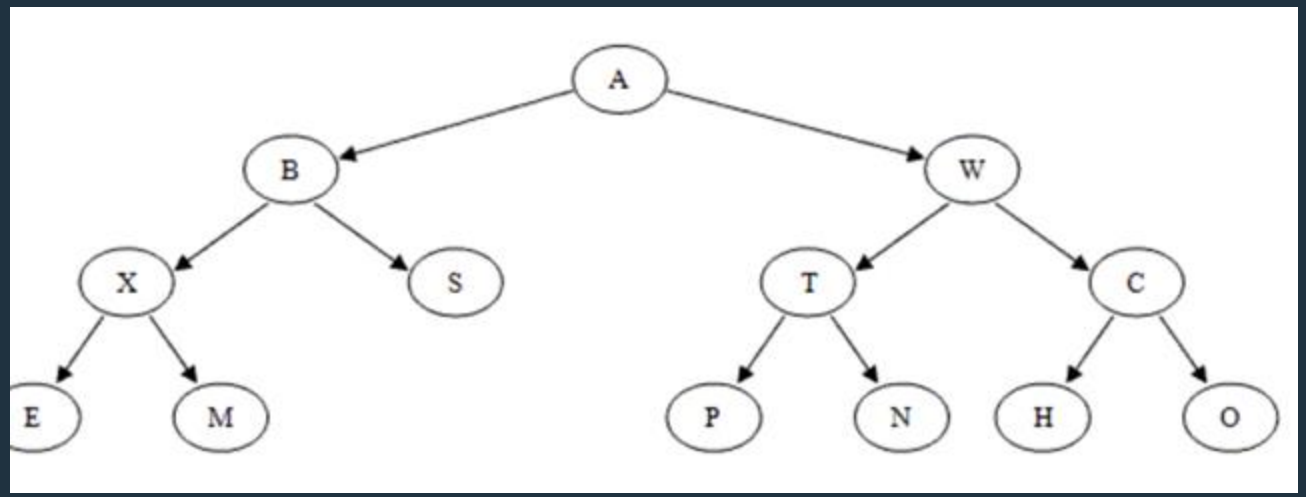
```java
public class Solution {
    public int mySqrt(int x) {
        int l = 0, r = x;
        int res = 0;

        while (l <= r) {
            int m = l + (r - l) / 2;
            if ((long) m * m > x) {
                r = m - 1;
            } else if ((long) m * m < x) {
                l = m + 1;
                res = m;
            } else {
                return m;
            }
        }

        return res;
    }
}
```

slidesmania.com

FRAMEWORK: BINARY TREE



Each root is said to be the parent of the roots of its subtrees.
Two nodes with the same parent are said to be siblings; they are the children of their parent.
The root node has no parent.

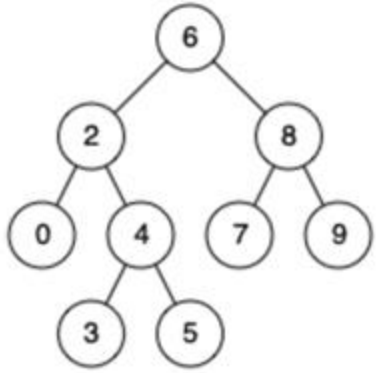
slidesmania.com

PROBLEM TWO: LOWEST COMMON ANCESTOR OF A BINARY SEARCH TREE

Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Example 1:



Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8
Output: 6
Explanation: The LCA of nodes 2 and 8 is 6.

slidesmania.com

PROBLEM TWO: LOWEST COMMON ANCESTOR OF A BINARY SEARCH TREE

Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Hint:

Compare Nodes with Current:

- If both nodes are greater than the current node, explore the right subtree.
- If both are smaller, explore the left subtree.
- If one is greater and the other is smaller, or if the current node is equal to one of the nodes, you've found the LCA.

CONSIDER RECURSION

slidesmania.com

PROBLEM THREE: Capacity to Ship Packages Within D Days

A conveyor belt has packages that must be shipped from one port to another within `days` days.

The `ith` package on the conveyor belt has a weight of `weights[i]`. Each day, we load the ship with packages on the conveyor belt (in the order given by `weights`). We may not load more weight than the maximum weight capacity of the ship.

Return the **least weight capacity** of the ship that will result in all the packages on the conveyor belt being shipped within `days` days.

Example 1:

Input: weights = [1,2,3,4,5,6,7,8,9,10], days = 5

Output: 15

Explanation: A ship capacity of 15 is the minimum to ship all the packages in 5 days like this:

1st day: 1, 2, 3, 4, 5

2nd day: 6, 7

3rd day: 8

4th day: 9

5th day: 10

Note that the cargo must be shipped in the order given, so using a ship of capacity 14 and splitting the packages into parts like (2, 3, 4, 5), (1, 6, 7), (8), (9), (10) is not allowed.

slidesmania.com

PROBLEM THREE: Capacity to Ship Packages Within D Days

A conveyor belt has packages that must be shipped from one port to another within `days` days.

The `i`th package on the conveyor belt has a weight of `weights[i]`. Each day, we load the ship with packages on the conveyor belt (in the order given by `weights`). We may not load more weight than the maximum weight capacity of the ship.

Return the **`least weight capacity`** of the ship that will result in all the packages on the conveyor belt being shipped within `days` days.

Hint:

We want to search for the capacity. Rather than brute-forcing each possible capacity, can we pick certain ones out to test and eliminate others?

slidesmania.com

```python
class Solution:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        l, r = max(weights), sum(weights)
        res = r

        def canShip(cap):
            ships, currCap = 1, cap
            for w in weights:
                if currCap - w < 0:
                    ships += 1
                    if ships > days:
                        return False
                    currCap = cap

                currCap -= w
            return True

        while l <= r:
            cap = (l + r) // 2
            if canShip(cap):
                res = min(res, cap)
                r = cap - 1
            else:
                l = cap + 1

        return res
```

slidesmania.com

TAKEAWAYS

01

Structure and Properties:

binary search tree (BST) is a binary tree where each node has at most two children, and for every node, all elements in its left subtree are smaller, and in its right subtree are larger. This property is crucial for efficient searching and sorting.

02

Traversal Techniques:

Be familiar with tree traversal methods: in-order, pre-order, and post-order. In-order traversal of a BST will give you elements in sorted order. Pre-order and post-order traversals are useful for tree reconstruction and hierarchical processes.

03

Time Complexity:

For a balanced BST, search, insertion, and deletion operations are generally O(log n). However, in the worst case (like a skewed tree), these operations can degrade to O(n).

04

Recursive Thinking:

Many tree problems are naturally suited for recursive solutions due to the tree's hierarchical structure. Understanding recursion and how to handle base cases is key to solving these problems effectively.

05

06

slidesmania.com